ONBRD-05: Deploying OneAgent

> **Series:** ONBRD — Dynatrace Onboarding | **Notebook:** 5 of 10 | **Created:** December 2025 | **Last Updated:** 07/30/2026

Getting Data Into Dynatrace

OneAgent is the foundation of Dynatrace monitoring. This notebook covers deployment strategies, installation methods, and verification steps to ensure your infrastructure is reporting data.

Table of Contents

Prerequisites

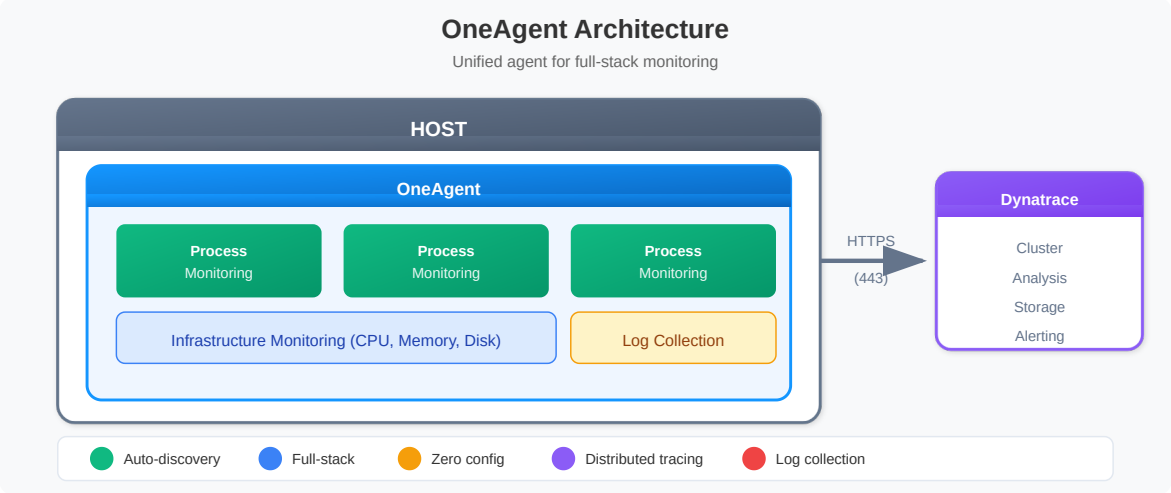
- Admin access to Dynatrace environment
- API token with `InstallerDownload` scope (created in ONBRD-02)
- Root/admin access to target hosts
- Network access from hosts to Dynatrace (port 443) or ActiveGate (ONBRD-03)

- Cloud integrations configured (ONBRD-04) for full context

1. What is OneAgent?

OneAgent is Dynatrace's unified monitoring agent that:

Capability	Description
Auto-discovery	Automatically detects hosts, processes, services
Full-stack	Infrastructure, application, and user experience
Zero configuration	Works immediately after installation
Distributed tracing	End-to-end transaction visibility
Log collection	Collects and forwards log data



2. Deployment Strategy

Phased Rollout (Recommended)

Don't deploy everywhere at once. Use a phased approach:

Phase	Scope	Purpose
Pilot	2-5 non-critical hosts	Validate installation, network, discovery
Expand	One application tier	Test service detection, tracing
Production	Full environment	Complete coverage

Deployment Methods by Environment

Environment	Recommended Method
Bare metal/VMs	Direct install or package manager
Kubernetes	Dynatrace Operator
OpenShift	Dynatrace Operator
AWS ECS	Task definition sidecar
Azure	VM Extension or AKS Operator
GCP	Direct install or GKE Operator

Network Requirements

OneAgent needs outbound HTTPS (443) to:

- `*.apps.dynatrace.com` (SaaS)

If direct access isn't possible, OneAgents connect through **ActiveGate** (see ONBRD-03).

3. Generating a Deployment Token

You need an API token with installer download permissions.

Location: Account Management → Access tokens → Generate new token

Required Scope

Scope	API Name	Purpose
PaaS integration - Installer download	`InstallerDownload`	Download OneAgent installer

Token Naming Convention

Use descriptive names:
- `prod-oneagent-deployment`
- `dev-installer-token`
- `k8s-operator-token`

> **Important:** Copy the token immediately after creation. It won't be shown again.

4. Installation Methods

Linux (Direct Download)

```
# Download the installer
wget -O Dynatrace-OneAgent.sh \
  "https://{tenant-
id}.apps.dynatrace.com/api/v1/deployment/installer/agent/unix/default/latest?
Api-Token={token}&amp;arch=x86&amp;flavor=default"

# Run the installer
sudo /bin/sh Dynatrace-OneAgent.sh
```

Linux (One-liner)

```
wget -O Dynatrace-OneAgent.sh "https://{tenant-
id}.apps.dynatrace.com/api/v1/deployment/installer/agent/unix/default/latest?
Api-Token={token}" &amp;&amp; sudo /bin/sh Dynatrace-OneAgent.sh
```

Windows (PowerShell)

```
# Download the installer
Invoke-WebRequest -Uri "https://{tenant-
id}.apps.dynatrace.com/api/v1/deployment/installer/agent/windows/default/lates
t?Api-Token={token}" -OutFile Dynatrace-OneAgent.exe

# Run the installer
.\Dynatrace-OneAgent.exe
```

> **Sprint 1.338 Windows network insight (Npcap):** new OneAgent Windows builds use **Npcap** for network-monitoring capture (replacing legacy WinPcap). Plan an Npcap install / redistribution into your Windows golden image when rolling out OneAgent on hosts that need network insight. See the OneAgent release notes for the affected version range.

Setting Primary Tags at Install Time (Sprint 1.337+)

Set `dt.security_context`, environment, team, cost-center, and other primary tags at OneAgent install time using `oneagentctl --set-host-tag`. Primary tags emit at the source on every signal (metrics, spans, logs, events) and feed directly into OpenPipeline routing, bucket assignment, and IAM scoping — more efficient than view-time auto-tagging.

```
# Set primary tags after install (Linux)
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl --set-host-tag
environment=prod
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl --set-host-tag
team=payments
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl --set-host-tag
dt.security_context=team-payments

# Set host group at install time (preferred — avoid renaming later)
sudo /bin/sh Dynatrace-OneAgent.sh --set-host-group=prod-app-payments
```

Decide your primary-tag and host-group taxonomy *before* the first install — retrofitting breaks history-based thresholds, IAM scoping, and any automation keyed on those values. See **FAQ-01 (host group naming strategy)** and **FAQ-02 (tagging sources, standards, strategy)** for the canonical guidance.

Configuration Management

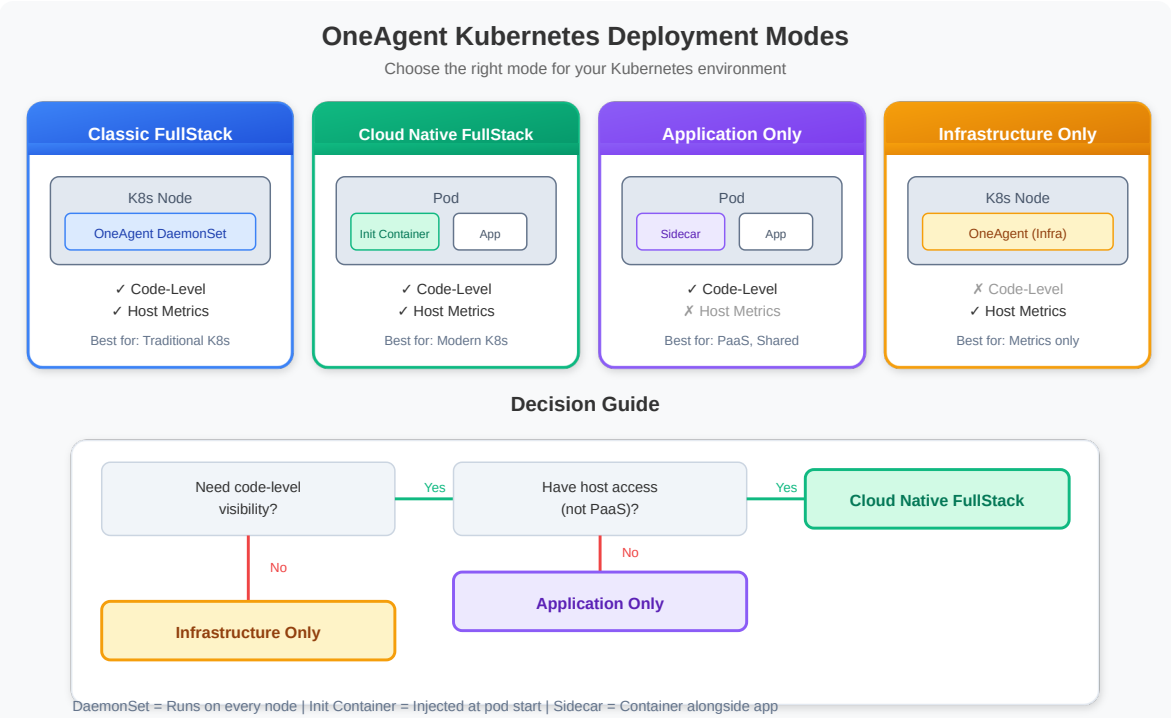
For Ansible, Puppet, Chef, or other tools, see:
- Ansible Collection
- Puppet Module
- Chef Cookbook

5. Kubernetes Deployment

For Kubernetes environments, use the Dynatrace Operator—the recommended approach for deploying and managing OneAgent in containerized environments.

OneAgent Deployment Modes

The Dynatrace Operator supports multiple deployment modes. Choose based on your requirements:



Mode	Deployment	Code-Level	Host Metrics	Best For
Cloud Native FullStack	Init container + CSI	Yes	Yes	Modern K8s (recommended)
Application Only	Sidecar	Yes	No	PaaS, shared nodes, OpenShift
Host Monitoring	DaemonSet	No	Yes	When code-level monitoring not needed

> **Note:** Classic FullStack mode is **not recommended for new deployments**. Dynatrace recommends Cloud Native FullStack for all Kubernetes environments.

Deployment Method: Helm vs Manifests

Method	Pros	Cons	Best For
Helm Charts	Version management, easy upgrades, values override	Requires Helm	Production, GitOps
Manifest Files	Simple, no dependencies	Manual version tracking	Air-gapped, constrained environments

Prerequisites

- `kubectl` access to cluster
- API token with required scopes
- Cluster admin permissions

Required Token Scopes for Kubernetes

Scope	Purpose
`InstallerDownload`	Download OneAgent
`entities.read`	Read topology
`settings.read`	Read configuration
`settings.write`	Write configuration
`DataExport`	Export data (optional)

Installation Option 1: Helm (Recommended)

```
# Add Dynatrace Helm repository
helm repo add dynatrace https://raw.githubusercontent.com/Dynatrace/dynatrace-operator/main/config/helm/repos/stable
helm repo update

# Create namespace and secret
kubectl create namespace dynatrace
kubectl -n dynatrace create secret generic dynakube --from-literal="apiToken=&lt;API_TOKEN&gt;" --from-literal="dataIngestToken=&lt;DATA_INGEST_TOKEN&gt;"

# Install with Helm
helm install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --set installCRD=true
```

Installation Option 2: Manifest Files

1. **Navigate to Kubernetes monitoring**
   - Infrastructure → Kubernetes → Add cluster

2. **Follow the wizard**
   - Select your distribution
   - Choose observability options
   - Download dynakube.yaml

3. **Apply to cluster**
   `bash kubectl apply -f https://github.com/Dynatrace/dynatrace-operator/releases/download/v1.10.1/kubernetes.yaml kubectl apply -f dynakube.yaml`

> **Pin a version you have validated.** The URL above is pinned rather than tracking a moving `latest` on purpose — a manifest install is the path you reach for precisely when you need reproducibility (air-gapped, GitOps, change-controlled clusters). `1.10.1` (published 07/22/2026) is the current recommendation. **Skip `1.10.0`:** its own release notes advise waiting for `1.10.1`, and GitHub now marks the `v1.10.0` release a prerelease — the machine-readable trace of that advice. `1.10.2` exists (published 07/30/2026) but ships without a changelog, so there is nothing yet to validate a bump against; move to it once its notes publish. Whatever you pin, validate it in a non-production cluster before it reaches the rest of the fleet, and check the Dynatrace Operator releases page for the version current when you read this.

4. **Verify deployment**
   `bash kubectl get pods -n dynatrace`

DynaKube Custom Resource Configuration

The DynaKube CR is the primary configuration for the Dynatrace Operator. Here are examples for each deployment mode:

> **Important:** Use `apiVersion: dynatrace.com/v1beta6` for new DynaKubes (`v1beta5` remains accepted). Operator **1.9.0** removed `v1beta3` from the CRD (*"Applying DynaKube resources using this version will fail"*), and Operator **1.10.0** (July 15, 2026) deprecates `v1beta4` — check `kubectl get dynakube -A -o jsonpath='{.items[*].apiVersion}'` before upgrading the Operator. v1beta6 adds OTLP exporter configuration.

Cloud Native FullStack (Recommended for most K8s):

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Cloud Native FullStack - automatic init container injection
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
      args:
        - --set-host-group=my-k8s-cluster

  # ActiveGate for Kubernetes API monitoring
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api
```

Application Only (for PaaS or shared nodes):

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Application Only - sidecar injection, no host monitoring
  oneAgent:
    applicationMonitoring:
      useCSIDriver: true
```

Host Monitoring (host metrics only):

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Host monitoring only - no code-level visibility
  oneAgent:
    hostMonitoring: {}
```

Air-Gapped / Private Registry Deployment

For environments without internet access, host images in a private registry:

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Custom image locations for air-gapped environments
  oneAgent:
    cloudNativeFullStack:
      image: my-registry.example.com/dynatrace/oneagent:latest

  activeGate:
    capabilities:
      - kubernetes-monitoring
    image: my-registry.example.com/dynatrace/dynatrace-activegate:latest
```

Image Mirroring Script:

```
# Mirror required images to private registry
# public.ecr.aws is the documented registry for Dynatrace component images.
# Pin the Operator to the version you validated above - not :latest.
IMAGES=(
  "public.ecr.aws/dynatrace/dynatrace-operator:v1.10.1"
  "public.ecr.aws/dynatrace/dynatrace-oneagent:latest"
  "public.ecr.aws/dynatrace/dynatrace-activegate:latest"
)

for img in "${IMAGES[@]}"; do
  docker pull $img
  docker tag $img my-registry.example.com/${img#*/}
  docker push my-registry.example.com/${img#*/}
done
```

Kubernetes Metadata and Dynatrace Tags

Dynatrace can leverage Kubernetes labels and annotations for observability:

Feature	Behavior	Limitations
Pod labels	Appear as `[Kubernetes]` context tags on processes	Requires OneAgent code module injection
Annotations	Available as custom metadata at Process level	Requires view RBAC permissions
Namespace labels	Available via Primary Grail tags	Does not enrich K8s metrics or events

Requirements:
- Service accounts need `view` access via rolebinding/clusterrolebinding
- Labels are read via Kubernetes REST API at deployment time
- Not all entities receive automatic tags (namespaces, pods, workloads have limited support)

For more control, configure automatic tagging rules: **Settings → Automatically applied tags**

Multi-Tenant / Namespace Isolation

For large clusters with multiple teams, use namespace selectors:

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: team-a-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://team-a-tenant.live.dynatrace.com/api

  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-a
```

This allows different teams/tenants to monitor different namespaces within the same cluster.

6. Verifying Deployment

After installation, verify OneAgent is reporting data.

```
// Check all hosts with OneAgent
fetch dt.entity.host
| fields entity.name, state, monitoringMode
| filter state == "RUNNING"
| sort entity.name
| limit 50

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only fields state / monitoringMode, which have NO Smartscape node
equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state / monitoring-mode detail.
// Other fields do map: entity.name -&gt; name.
```

```
// Check hosts by monitoring state
fetch dt.entity.host
| summarize host_count = count(), by: {state, monitoringMode}
| sort host_count desc

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only fields state / monitoringMode, which have NO Smartscape node
equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state / monitoring-mode detail.
```

```
// Check hosts by monitoring state - useful for verifying deployment
fetch dt.entity.host
| fields entity.name, state, monitoringMode
| summarize host_count = count(), by: {state}
| sort host_count desc

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only fields state / monitoringMode, which have NO Smartscape node
equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state / monitoring-mode detail.
// Other fields do map: entity.name -&gt; name.
```

```
// Check discovered services
fetch dt.entity.service
| fields entity.name, serviceType
| summarize service_count = count(), by: {serviceType}
| sort service_count desc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | fields name, dt.service.sdv1_type
//   | summarize service_count = count(), by: {dt.service.sdv1_type}
//   | sort service_count desc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: serviceType -&gt; dt.service.sdv1_type; entity.name -&gt; name.
```

```
// Check for processes discovered
fetch dt.entity.process_group
| fields entity.name
| sort entity.name
| limit 50

// Smartscape note (dt.entity.* is deprecated but still functional):
Smartscape models
// individual processes, not process GROUPS — smartscapeNodes "PROCESS" is a
different
// granularity (process instances), so its results are not comparable to a
process-group
// query. Keep the classic dt.entity.process_group query above.
```

Host Verification Commands

Linux:

```
# Check OneAgent status
sudo systemctl status oneagent

# Check connection to Dynatrace
sudo /opt/dynatrace/oneagent/agent/tools/oneagent-connection-check

# Check OneAgent logs
sudo tail -100 /var/log/dynatrace/oneagent/oneagent.log
```

Windows:

```
# Check service status
Get-Service -Name "Dynatrace OneAgent"

# Check connection
&amp;amp; "C:\Program Files\dynatrace\oneagent\agent\tools\oneagent-connection-
check.exe"
```

What You'll See on Modern Tenants

Once OneAgent is reporting and services appear in the verification queries above, **Enhanced Endpoints for SDv1** (Dynatrace v1.329+) shapes what shows up under each service:

Tenant creation date	Default state	What you do
v1.333 or later	**Always on, not configurable**	Nothing — each service automatically exposes individual endpoints with `dt.service.request.*` metrics
v1.330 – v1.332	On by default (toggleable)	Verify the toggle hasn't been disabled
v1.329 or earlier	Off by default	Enable at *Settings* → *Process and contextualize* → *Services* → *Service detection v1* if you want per-endpoint metrics

Why it matters during onboarding: before Enhanced Endpoints, only manually marked "key requests" emitted individual metrics — everything else collapsed into a single `NON_KEY_REQUESTS` bucket. Now every detected endpoint is named and tracked automatically, which materially changes what's visible on day one.

Services not affected: external services, background activity, queue listeners, key-value stores — these never get per-endpoint metrics regardless of the setting.

> **Sources:** Enhanced endpoints for SDv1 (DT docs).

7. Troubleshooting

Common Issues

Issue	Cause	Solution
Host not appearing	Network blocked	Check firewall rules for 443
Host showing but no data	Agent not running	Restart OneAgent service
Services not discovered	Processes not restarted	Restart monitored applications

Issue	Cause	Solution
Old OneAgent version	Auto-update disabled	Enable auto-update or manual upgrade
Connection errors	Proxy required	Configure ActiveGate or proxy

Network Verification

```
# Test connectivity to Dynatrace
curl -v https://{tenant-id}.apps.dynatrace.com/api/v1/time

# Check DNS resolution
nslookup {tenant-id}.apps.dynatrace.com

# Test with OpenSSL
openssl s_client -connect {tenant-id}.apps.dynatrace.com:443
```

Process Restart Requirement

OneAgent injects into running processes. After initial installation:

- **New processes** - Automatically monitored
- **Existing processes** - Restart required for full monitoring

For deep code-level visibility, restart:
- Application servers (Tomcat, JBoss, etc.)
- Web servers (Apache, Nginx, IIS)
- .NET/Java applications
- Node.js applications

8. Next Steps

With OneAgent deployed and verified:

1. **ONBRD-06: Organizing Your Environment** — Set up tags, segments, naming conventions, and `dt.security_context`
2. Wait 15-30 minutes for full topology discovery
3. Check Smartscape for service dependencies
4. Plan broader rollout based on pilot results

Where to Go Deeper

- **K8S series** (15 notebooks) — Cluster monitoring, DynaKube depth, GitOps for K8s deployments
- **FAQ-01** — Host group naming strategy (decide before first install)
- **FAQ-02** — Tagging sources, standards, and strategy (primary tags vs auto-tags vs cloud tags)

Deployment Checklist

- [ ] Pilot hosts deployed (2-5 hosts)
- [ ] Hosts appearing in Dynatrace
- [ ] OneAgent status "RUNNING" on all hosts
- [ ] Services being discovered
- [ ] Connection check passing
- [ ] Smartscape showing topology
- [ ] Host groups set at install time (not retrofitted later)
- [ ] Primary tags (environment, team, `dt.security_context`) emitted at source
- [ ] Windows hosts: Npcap install path planned (sprint-1.338 dependency)

Summary

In this notebook, you learned:

- What OneAgent does and how it works
- Phased deployment strategy
- How to generate deployment tokens
- Installation methods for Linux, Windows, and Kubernetes
- Setting primary tags and host groups at install time (sprint-1.337+ pattern)
- Windows Npcap requirement (sprint-1.338)
- How to verify successful deployment
- Common troubleshooting steps

References

- OneAgent Overview
- Linux Installation
- Windows Installation
- Kubernetes Operator
- Deployment API
- OneAgent Attribute Enrichment

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.